

5SC28 - Design Project Final report

Nemo ter Bogt - 2170677

Dara Prins - 1732978

Alecsandru Kreefft-Libiu - 2371723

Illya Averchenko - 2055961

I. NEURAL NETWORK SYSTEM IDENTIFICATION

A. Approach

As stated in practical exercise set 3, Problem 2, a common approach for choosing a neural network structure is to:

- Look up what others have used on similar problems and use that as a starting point.
- Conduct a hyperparameter search using cross-validation around that starting point.

Hence, rather than empirically exploring a large set of architectures, we started from a short literature survey.

A limitation noted by Dong et al. is that “one limitation of the typical recurrent layer is that the time window of its inputs and outputs is required to be fixed during the training” [1]. Since our training data uses a fixed window of 15 steps, this constraint does not affect us, and we therefore adopted a Recurrent Neural Network (RNN) and its variant with Long Short Term Memory (LSTM).’

Additionally, a simple multilayer perceptron (MLP) network was also adopted as a baseline. In a MLP, the input information only passes through hidden layers into an output layer. Since it does not have any memory, time-dependent behavior must be supplied in the input information. For this we use the fixed 15 step window from the training data. The MLP receives the 15 previous timesteps as a fixed length input vector and predicts the next disk angle. The main reason for using an MLP is that it provides a simple and clear baseline for the ANN-based modeling task. It is easier to train and interpret than recurrent models, because it has no feedback connections or memory mechanism. This makes the MLP results a useful reference for LSTM and RNN results.

Plain RNNs are known to struggle to learn long-term dependencies through gradient descent [2], a shortcoming that LSTMs were specifically designed to address through their gated memory mechanism [3]. We therefore also train an LSTM and assess whether, at a lookback of 15 steps, the LSTM structure captures long-term dynamic relationships more effectively than the RNN.

The remainder of this section covers data preparation (Section I-B), model definition (Section I-C), model training (Section I-D), and simulation/evaluation results (Section I-E).

B. Data Preparation

The raw benchmark data is transformed into a supervised learning problem using a sliding-window formulation. For each time step, the preceding $w = 15$ samples pairs of input

and output θ) are stacked into an input window, and the output θ at the current step is taken as the one-step-ahead prediction target. This autoregressive framing exposes the network to recent input history as well as past outputs, consistent with the NARX modelling assumption.

The resulting samples are split chronologically rather than randomly, so as to preserve the temporal ordering of the time series and to avoid leaking future information into the training set. The first 80% of the windowed samples are used for training and the final 20% are held out as a test set. From the training portion, the most recent 10% is set aside as a validation set for model selection and early stopping.

All features are standardized to zero mean and unit variance. The scaler is fitted *only* on the training features, and the same fitted statistics are then applied to the validation and test sets; this prevents data leakage from the held-out data into the normalization step. Furthermore, the prediction target is standardized using the training mean and standard deviation of the output channel θ (the final feature column), at evaluation time.

C. Network Architecture

The feedforward Multilayer Perceptron consists of an input layer with a width of 30, for the 15 time steps where both the motor voltage and disk angle are given. After the input are hidden, fully connected, layers, the depth of which is determined during the optimization. This also holds for the width of these hidden layers. Finally it outputs a single value for the disk angle. Between each layer is a non-linear activation function, which allow the model to estimate the non-linear relation between motor voltage, current disk angle and future disk angle. The specific activation function is also determined in the optimization of the model.

For the recurrent architectures, two types are implemented and compared: a Long Short-Term Memory (LSTM) network [3] and a recurrent neural network (RNN). Both share the same overall structure: a recurrent encoder that processes the 15-step input window, followed by a multi-layer perceptron (MLP) output head that produces the one-step-ahead prediction. Let the input window be $\mathbf{x}_{1:15}$, with each step $\mathbf{x}_t \in \mathbb{R}^2$ comprising the input–output pair (u_t, θ_t) , and let H denote the hidden-state size. Both models operate over a fixed look back of 15 steps.

For the activation function both TanH and sigmoid were considered. However as maximum derivative of TanH (1.0) is 4 times larger than that of sigmoid (0.25) and a RNN more

sensitive to vanishing gradients, we use TanH to get a more robust comparison between the models.

For both models the number of hidden layers H_n , hidden layer size, batch size, learning rate, Hidden to output nodes ($H2O$) and learning rate are tunable parameters.

D. Optimization

For obtaining the final models that are evaluated, we have implemented hyperparameter optimization using the Optuna module. During each Optuna study, parameter combinations are sampled from the discrete and continuous hyperparameter value domains defined in Table I. After each trial the Tree-structured Parzen Estimator (TPE) sampler is conditioned on the performance of the previously evaluated combinations, which can offer advantages over uninformed methods such as grid search and random search [4]. Both studies (LSTM and RNN) consisted of 100 trials with a maximum of 400 epochs per trial; a seeded TPE sampler combined with a median pruner ($n_startup_trials=5$, $n_warmup_steps=2$) was used, terminating unpromising trials early. Each trial was scored by the free-running simulation NRMSE (%) on the validation sequence, which the study minimised. The resulting best parameter set for each study is reported in Table II. Finally, a model with the best parameters was retrained for 400 epochs.

TABLE I: Hyperparameter search domains used in the LSTM and RNN Optuna studies. Identical domains were used for both studies.

Hyperparameter	Type	Domain	Sampling
Hidden size	Integer	[10, 100]	Uniform
Number of layers	Integer	[1, 5]	Uniform
Output net width	Integer	[10, 100]	Uniform
Learning rate	Float	$[10^{-4}, 10^{-2}]$	Log-uniform
Batch size	Categorical	{32, 64, 128}	—

TABLE II: Best hyperparameter set found by each Optuna study (100 trials), with the corresponding validation simulation NRMSE.

Hyperparameter	LSTM	RNN
Hidden size	87	58
Number of layers	4	5
Output net width	89	69
Learning rate	1.02×10^{-4}	1.53×10^{-4}
Batch size	128	64
Validation NRMSE (%)	3.12	3.90

E. Evaluation

The evaluation on the test set (Table III) shows that the LSTM outperforms both benchmarks provided in the assignment, as well as the RNN, achieving the lowest error across all metrics. The RNN outperforms the linear benchmark and is essentially on par with the Good NN benchmark, trailing it only marginally. The MLP does outperform the linear benchmark, but is still worse than the LSTM, RNN and the

TABLE III: Evaluation on the test set (one-step-ahead prediction).

Model	RMSE (rad)	RMSE (deg)	NRMS (%)
MLP	0.00557	0.319	1.08
LSTM	0.00302	0.173	0.58
RNN	0.00392	0.225	0.76
Good NN (bench)	0.00382	0.219	—
Linear (bench)	0.00665	0.381	—

TABLE IV: Evaluation in the simulation scenario.

Model	RMSE (rad)	RMSE (deg)	NRMS (%)
MLP	0.06203	3.554	12.00
LSTM	0.02857	1.637	5.53
RNN	0.06938	3.975	13.42
Lower bound	0.01950	1.120	—
Good NN (bench)	0.02710	1.550	—
Linear (bench)	0.25500	14.600	—

good NN benchmark. Since the MLP does not have any recurrent memory and is a relatively simple network, it is expected to perform worse than the RNN and LSTM.

In the simulation scenario (Table IV), the LSTM again substantially outperforms both the linear benchmark and the RNN, and now comes very close to the Good NN benchmark (0.02857 rad versus 0.02710 rad), with both models remaining above the lower bound. It nonetheless still narrowly fails to surpass the Good NN benchmark. We therefore use the LSTM to fill in the missing values in the solution files. The MLP performance decreases more strongly, with an RMSE of 0.0620 rad. This is expected because simulation is a more difficult task: the model recursively uses its own previous predictions, causing errors to accumulate over time. Since the MLP has no internal recurrent state and only uses a fixed-size input window, it is less suitable for capturing long-term dynamic behaviour than the LSTM. It still performs better than the linear benchmark, showing that it captures the system dynamics better than a linear system would.

The remaining gap in simulation can be explained by the model being trained as a one-step-ahead predictor: it is not fully robust against its own prediction errors being fed back as inputs. The gap is now small, however, suggesting that the LSTM generalises well under recursive prediction; closing it entirely would likely require reconsidering the network architecture or the training objective (e.g. multi-step / free-run training).

II. GAUSSIAN PROCESSES

For the Gaussian Process part, we implemented a GP-NARX model for one-step-ahead prediction and free-run simulation of the unbalanced disk. The goal was to identify the nonlinear input-output dynamics of the system from the measured motor voltage u and disk angle θ . Compared with the neural-network models, the Gaussian Process produces both a mean prediction and a predictive uncertainty and thus this makes it useful not only as a nonlinear regression model, but also as a probabilistic baseline for system identification.

A. Model structure

The model is formulated as a nonlinear autoregressive model with exogenous input (NARX). The next disk angle is predicted from a finite history of previous inputs and outputs. At each time step k , the regressor vector is

$$x_k = [u_{k-n_b} \quad \cdots \quad u_{k-1} \quad \theta_{k-n_a} \quad \cdots \quad \theta_{k-1}]^T, \quad (1)$$

and the prediction target is

$$y_k = \theta_k. \quad (2)$$

Here, n_a is the number of past output samples and n_b is the number of past input samples. Several NARX orders were tested. The tested configurations were $n_a = n_b = 5, 6, 7$, and 10. The best-performing model was obtained with $n_a = 10$ and $n_b = 10$, meaning that the GP receives the previous ten voltage samples and the previous ten angle samples as input. This gives a 20-dimensional regression problem.

The use of past outputs is important because the disk angle at the next time step is not determined only by the current motor input, it also depends on the recent dynamic evolution of the mechanical system. Therefore, the NARX structure provides the GP with a compact representation of the system state history.

B. Gaussian Process regression

The GP was trained using a constant kernel multiplied by a radial basis function (RBF) kernel, plus a white-noise term,

$$k(x, x') = \sigma_f^2 \exp \left(-\frac{1}{2} \sum_{j=1}^d \frac{(x_j - x'_j)^2}{\ell_j^2} \right) + \sigma_n^2 \delta_{x, x'}. \quad (3)$$

An automatic relevance determination version of the RBF kernel was used, so each lagged input and output has its own length-scale ℓ_j . This is useful because not all previous inputs and outputs are equally informative. A short length-scale indicates that the prediction is sensitive to that specific regressor, whereas a long length-scale suggests that the corresponding lag is less relevant.

Since GP training becomes computationally expensive for large datasets, the GP was trained on a subset of the available NARX samples rather than on the full dataset. This allowed several NARX orders to be compared while keeping the training time manageable.

C. Evaluation approach

The GP was evaluated in two different ways. First, it was tested as a one-step-ahead predictor. In this setting, the GP predicts the next angle using the true previous measured angles. This is the easier evaluation setting because the input history always contains measured data.

Second, the GP was evaluated in free-run simulation. In this setting, after the initial history, the model recursively feeds its own previous predictions back into the input vector. This is more difficult and more relevant for system simulation,

TABLE V: Comparison of GP-NARX models with different autoregressive orders.

Order	Time [s]	Pred. RMSE [rad]	Pred. NRMS [%]	Sim. RMSE [rad]	Sim. NRMS [%]
(5, 5)	200.4	0.003541	0.68	0.036455	7.05
(10, 5)	216.3	0.003623	0.70	0.042328	8.18
(10, 7)	207.5	0.003709	0.72	0.043719	8.45
(10, 10)	261.6	0.003834	0.74	0.044018	8.51
(7, 10)	216.9	0.003761	0.73	0.044291	8.56
(7, 7)	188.1	0.005451	1.05	0.048239	9.33
(6, 6)	260.7	0.005883	1.14	0.053025	10.25
(5, 10)	181.9	0.006016	1.16	0.058976	11.40

because prediction errors can accumulate over time. Therefore, the free-run simulation results can show whether the learned model remains useful when rolled out over some time.

The results in Table V show that increasing the NARX order does not automatically improve the GP model. The best result is obtained with the relatively compact $(n_a, n_b) = (5, 5)$ model. This model achieves the lowest one-step prediction RMSE of 0.003541 rad, and the lowest free-run simulation RMSE of 0.036455 rad.

TABLE VI: Best GP-NARX validation result for $(n_a, n_b) = (5, 5)$.

Evaluation setting	RMSE [rad]	RMSE [deg]	NRMS [%]
One-step prediction	0.003541	0.203	0.68
Free-run simulation	0.036455	2.089	7.05

This suggests that, for the available amount of GP training data, adding more delayed inputs and outputs does not necessarily help. Higher-order models increase the dimensionality of the regression problem, which can make GP learning more difficult when only a limited subset of samples is used for training. The asymmetric configurations also show that simply increasing the voltage history is not beneficial: the (5, 10) model gives the worst free-run simulation result. This indicates that the most useful information is already contained in a shorter recent input-output history.

Figure 1 shows a zoomed free-run simulation interval for the best GP-NARX model. The measured and simulated trajectories almost overlap in the upper plot, which indicates that the model captures the dominant oscillatory behaviour of the disk. Since the difference between the two trajectories is difficult to see directly, the lower plot shows the signed simulation error $\hat{\theta} - \theta$. The error remains small for most of the interval, with larger deviations appearing around rapid changes in the disk motion.

The one-step prediction result show that the GP-NARX model captures the local input-output behaviour of the disk quite accurately. The free-run simulation error is larger, expectedly, because the model must rely on its own previous predictions after the initial history (small errors are recursively fed back into the model and can accumulate). Nevertheless, the free-run RMSE of 0.036455 rad remains relatively low, indicating that the GP captures the nonlinear dynamics of the disk.

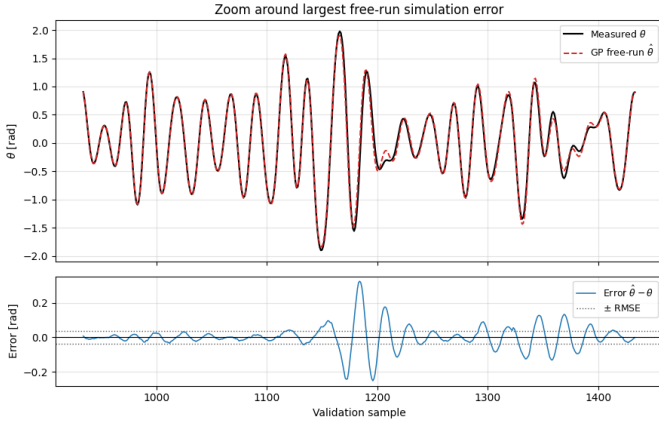


Fig. 1: Zoomed free-run simulation of the best GP-NARX model. The upper plot compares the measured disk angle with the recursively simulated GP trajectory. The lower plot shows the signed simulation error $\hat{\theta} - \theta$, together with the $\pm$ RMSE reference lines.

III. Q-LEARNING

For the Q-Learning part, we implemented a tabular version of Q-learning and a Deep Q-Network(DQN) to control the input voltage of the motor to balance the disk at the top.

A. Model environment

The environment is based on a digital simulation of the unbalanced disk, implemented as a custom Gymnasium environment. To facilitate learning and avoid issues with angle wrapping, the observation space consists of the sine and cosine of the disk angle as well as the normalized angular velocity: $s = [\sin(\theta), \cos(\theta), \omega]$. Small Gaussian noise is added to the angle and velocity observations to simulate sensor inaccuracies and improve the robustness of the trained policy.

The motor input was capped at ± 3.0 V to match the physical limits of the setup. For Q-learning, this continuous action range was discretized into 11 evenly spaced voltage values. The system dynamics are integrated using a Runge-Kutta solver with 0.025 seconds between steps, truncating after 200 steps (about 5 seconds).

B. Model structure

For the tabular Q-learning algorithm, an ϵ -greedy approach was used to balance exploration and exploitation. The agent will usually take the action with the highest expected reward. However, when the random value between 0 and 1 is lower than ϵ , it will choose a random action. This exploration is necessary so the agent can discover better actions instead of only exploiting known Q-values. The learned values are stored in a Q-table, where each value represents the expected cumulative future reward for taking a certain action in a certain state. Each iteration the Q-value gets updated based on the reward and the best expected reward in the next state, using the following rule:

$$Q(x_k, u_k) = Q(x_k, u_k) + \alpha \left[r_{k+1} + \gamma \max_u Q(x_{k+1}, u) - Q(x_k, u_k) \right] \quad (4)$$

Here $Q(x_k, u_k)$ is the current Q-value for the state x_k and the action u_k . α is the learning rate, which determines the rate at which the new information impacts new Q-value. r_{k+1} is the reward after the action and γ is the discount factor. This determines how much future rewards impact the new Q-value. This future reward is represented in the equation by $\max_u Q(x_{k+1}, u)$. This selects the highest expected future reward over all possible actions in the next state.

In the real setup, the states and actions are continuous, and since in tabular Q-learning the Q-values are stored in a table, both the action and state spaces must be discretized. This was done using two wrappers, which discretize both spaces. For the observation space this was done by using the upper and lower limit of the continuous space in combination with a given amount of bins. It then categorizes the input in the corresponding discrete state. The discretization of the action also uses a minimum and maximum, between which it creates evenly spaced action values. This discretization is necessary for tabular Q-learning, but it has some limitations. Too few bins and the model becomes too simple, and too many bins and the computation becomes lengthy and the model becomes harder to train.

To overcome the limitations of state discretization, we implemented a Deep Q-Network (DQN). Instead of a table, a Multi-Layer Perceptron (MLP) is used to approximate the Q-values for the continuous states. The network architecture consists of an input layer for the 3-dimensional state, two hidden layers with 64 nodes, each using ReLU activation functions and a linear output layer which provide the Q-values for the 11 discrete actions.

To stabilize training, two key reinforcement learning mechanisms were utilized: a Replay Buffer that allows the model to remember and fall back on past experiences. It has a capacity of 100,000 transitions to break correlations in the training data, and a separate target network that is updated via soft updates ($\tau = 0.005$), to provide consistent targets.

C. Parameter tuning

In Reinforcement Learning, the agent acts as a strict optimizer seeking to maximize cumulative reward, often exploiting any loopholes in the reward formulation. Therefore, we applied specific reward shaping to carefully balance the conflicting objectives of reaching the top equilibrium, maintaining stability, and avoiding excessive wear on the physical motor. For tabular Q-learning, both the learning parameters α , γ and ϵ , as well as the discretization settings, have to be tuned. The observation space was represented using $\cos(\theta)$ and $\sin(\theta)$ and angular velocity ω , which were all divided into 30 bins. The action space was discretized into 11 different actions.

$$r = -(\cos(\theta) + 1) - 0.01\omega^2 \quad (5)$$

The angle dependent reward is bounded between 0 and -2 . The penalty for high angular velocity should improve the smoothness of the balancing. Assuming the Q-table is initialized with zeros, this strictly non-positive structure enforces *optimistic initialization*, naturally driving the agent to explore to avoid negative feedback. Furthermore, it relies solely on the immediate state variables (θ and ω). Introducing penalties based on previous actions (like u_{prev}) would require adding another dimension to the discretized state space, causing an exponential explosion in table size that makes training computationally intractable.

Circling back to DQN, a neural network handles high-dimensional inputs efficiently, allowing us to include u_{prev} to heavily shape the motor's behavior. Additionally, the DQN reward is mathematically shifted to provide $+1$ at the target equilibrium, as a distinct positive reward provides a more stable gradient during the network's backpropagation. The specific reward function formulated at each timestep is given by:

$$r = \cos(\theta - \pi) - 0.001\omega^2 - 0.01u^2 - 0.005(u - u_{prev})^2 \quad (6)$$

This equation is quite specific and consists of four distinct components guiding the agent's behaviour:

- **Positional Reward** ($\cos(\theta - \pi)$): provides the primary motivation for the swing-up task. It utilizes the cosine function to create a smooth and continuous gradient that gives $+1$ at the perfect upright position (when $\theta = \pi$) and -1 At the resting position ($\theta = 0$). This can otherwise be described as the agent being guided by the continuous gradient during early exploration, instead of simply having a sparse reward.
- **Velocity Penalty** ($-0.001\omega^2$): penalizes the agent for moving with (excessive) high angular velocity. Without this term, we noticed that one of the solutions the DQN came up with is spinning erratically and quickly through the top point, still gaining the reward. The coefficient is quite small to account for necessary small momentum changes when the disk has to spin up, and large enough to also decelerate in the top position once it reaches the target equilibrium.
- **Energy/Control Penalty** ($-0.01u^2$): This term penalizes the magnitude of the motor input voltage (u). By penalizing the square of the voltage, small inputs required to maintain balance are lightly penalized, while maximum control efforts (e.g., $\pm 3V$) are heavily punished. This encourages the agent to find the most energy-efficient trajectory, preventing excessive current draw and motor overheating when deployed on the physical setup.
- **Smoothing Penalty** ($-0.005(u - u_{prev})^2$): This term penalizes sudden, drastic changes in the control signal between consecutive timesteps. Because DQN selects from a discrete set of action bins, the agent might otherwise

attempt to maintain balance by rapidly alternating between opposing voltages. This high-frequency oscillation, known as *chattering*, can introduce severe mechanical vibrations and damage physical gearboxes.

During training, we used an ϵ -greedy exploration strategy, where the exploration rate ϵ decays exponentially from 1.0 down to a minimum of 0.05 over 40,000 steps. The network was optimized using the Adam optimizer with a learning rate of 2×10^{-4} , a discount factor $\gamma = 0.99$, and a batch size of 64. The model was trained for a total of 600 episodes.

In the case of the tabular Q-learning training, it was trained with 2,500,000 steps, with an epsilon decay from 0.9 to 0.01. The episode length was set to 200, so it was trained on 12,500 episodes. The resulting training reward can be seen in figure 2.

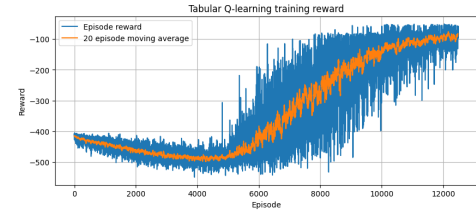


Fig. 2: Training reward over episodes for tabular Q-learning

D. Results

In the simulation setup the Tabular Q-learning policy successfully learned the swing-up and balancing behaviour. Figure 3 shows the disk quickly moving up and stabilizing near the top, while figure 4 shows that once at the top, the agent only makes small adjustments.

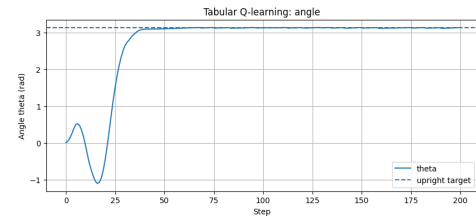


Fig. 3: The angle over 200 steps for tabular Q-learning

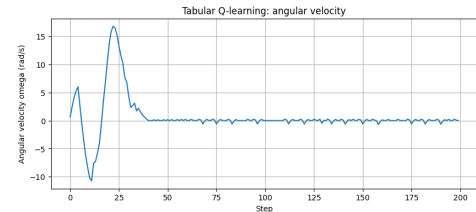


Fig. 4: The angular velocity over 200 steps for tabular Q-learning

When evaluated over 20 episodes with $\epsilon = 0$, the mean reward averages at around -80, which is a major improvement

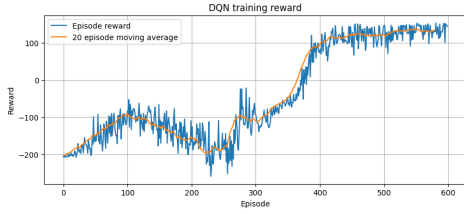


Fig. 5: Reward history of the DQN

from the -400 to -500 at which the policy starts. The average absolute angle error is roughly 23° . This might be due to the fact that it has to swing up in the beginning, and also due to the small movements it makes when balancing at the top. This is also reflected in the average angular velocity, which is 1.48 rad/s. This is relatively low and also influenced in the swing-up part of the episode.

The DQN also successfully learned the swing-up and balancing task. By analyzing the training history, the average episode rewards were initially negative but showed improvement in the following episodes, stabilizing at around 400 episodes until the end as seen in Figure 5. The agent received an average reward of about 140 during the whole period, seemingly indicating stable and successful behaviour.

There was also tests done on the physical setup, where we evaluated our model on a real-life setup. Unfortunately we were unable to successfully balance the disk on top due to time constraints. By the time the drivers were installed and the environment was set up, we had too little time left to optimize our models to balance on the actual setup. The simulation results discussed earlier were obtained after our session with the physical setup and we were not able to test the most recent implementations.

IV. ADVANCED REINFORCEMENT LEARNING: A2C AND PPO

For the advanced reinforcement learning part of the assignment we look at two on-policy actor-critic algorithms: Advantage Actor-Critic (A2C) and Proximal Policy Optimization (PPO). We use both as imported from the `stable-baselines3` (SB3) library.

A. Algorithms

A2C and PPO build on the same idea: an actor $\pi_\theta(a | s)$ is updated along an estimated policy gradient, and a critic $V_\phi(s)$ gives us the baseline we use to compute the advantage $\hat{A}_t$ (we use GAE for this).

The main difference is how much they squeeze out of each batch of collected data. A2C takes a single gradient step on a freshly collected rollout and then throws it away. PPO reuses each rollout for several epochs of minibatch updates, and to keep that reuse stable it optimizes a clipped objective, which limits how far the policy can move per update.

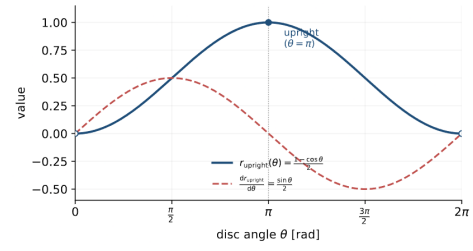


Fig. 6: The raised-cosine upright BaseR term $r_{\text{upright}}(\theta) = (1 - \cos \theta)/2$ (solid) and its derivative $\sin(\theta)/2$ (dashed).

We trained both models in the same environment and the same rewards, such that we can see directly what this difference does on our balancing task.

B. Reward Shaping

We started by designing a reward structure that was as simple as possible: the “upright” reward (*BaseR*), based on the disc angle θ ,

$$r_{\text{upright}}(\theta) = \frac{1 - \cos \theta}{2}, \quad (7)$$

where $\theta = \pi$ is the upright (target) position and $\theta = 0$ the resting position. The reward shape (Figure 6) is a *raised cosine*: it is smooth (C^∞), bounded to $[0, 1]$, and increases monotonically from 0 at $\theta = 0$ to its maximum of 1 at $\theta = \pi$. Its derivative is

$$\frac{dr_{\text{upright}}}{d\theta} = \frac{\sin \theta}{2}, \quad (8)$$

which is zero at both $\theta = 0$ and $\theta = \pi$ and largest at $\theta = \pi/2$. Looking at the derivative, the signal is strongest during the swing-up around $\theta = \pi/2$ and gives us very little gradient for the fine balancing close to $\theta = \pi$.

Therefore so it is harder for the agent to learn the added value of reaching higher, which requires spending more time in lower-reward segments to build up momentum. Hence we have tuned the reward function to add a sharp bonus around the top,

$$r(\theta) = \frac{1 - \cos \theta}{2} + 0.5 \exp\left(-\left(\frac{\Delta_{\text{top}}(\theta)}{0.25}\right)^2\right), \quad (9)$$

where

$$\Delta_{\text{top}}(\theta) = \text{atan2}(\sin(\theta - \pi), \cos(\theta - \pi)) \quad (10)$$

is the angular distance to the top, wrapped to $[-\pi, \pi]$.

The tuned rewards plot *TunedR* and its derivative are shown in Figure 7. The bonus is a Gaussian centered on the top ($\theta = \pi$) with a width of about 14° and a peak of 1.0, so the total reward rises to 1.5 at the upright position.

C. Episode Termination and the Truncation Condition

It is essential that the agent is exposed to both phases: those that require generating swing dynamics and those that require maintaining balance at the top, in order to achieve robust generalization across these regimes.

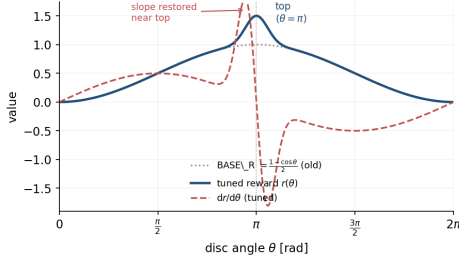


Fig. 7: The TunedR $r(\theta) = (1 - \cos \theta)/2 + 0.5 \exp(-(\Delta_{\text{top}}/0.25)^2)$ (solid) and its derivative (dashed), with the original BaseR shown for reference (dotted).

We found that introducing a truncation condition at 200 time steps provides an adequate compromise that facilitates progress toward both learning objectives.

D. Action Noise & Latency

In order to better mimic the real world limitations that the UnbalancedDisks motors have we have made implemented some assumptions to distinguish between models trained in the default and our altered *Robust environment*.

We suspect the actions outputted by the model are rarely executed at their exact values. Hence the *robust environment* adds white gaussian noise $N(0, 0.5)$ to the outputted actuator gain (before clipping). Furthermore the *Robust environment* simulated dropped actions by holding the previous torque with 5% chance.

E. Evaluation and Experimental Setup

We evaluate the learned controllers using the mean base-task return over 10 deterministic evaluation episodes. This value measures how well the agent performs the swing-up task and how long it can keep the disk close to the upright position. Since some PPO configurations reach the reward ceiling of approximately 180, final reward alone is not always enough to distinguish the best controllers. When several configurations reach this threshold, it is more useful to compare how many training steps they need to reach it.

For each algorithm, six configurations were tested. These configurations combine the reward formulation, observation representation, and disturbance setting. With raw observations $[\theta, \omega]$, only the nominal environment was tested, using both the base and tuned rewards. With sine-cosine observations $[\sin \theta, \cos \theta, \omega]$, both nominal and robust environments were tested, again with both reward functions. This gives six configurations for A2C and six configurations for PPO, resulting in 12 configurations in total. Each configuration was trained with six different random seeds. Therefore, the final experiment consists of $12 \times 6 = 72$ trained agents.

Using multiple seeds makes the comparison more reliable, because reinforcement learning training is stochastic. A policy can perform better or worse because of a favourable or unfavourable seed, rather than because the configuration itself is better. For this reason, the statistical comparison is performed

using matched seed blocks. Since each configuration is trained with the same six seeds, the results can be compared seed-by-seed. The main statistical test used is the paired Wilcoxon signed-rank test. Paired t-tests and Cohen’s d_z are also reported as supporting statistics.

TABLE VII: Overview of the 12 reinforcement learning configurations. Each configuration was trained with six random seeds.

Config.	Algorithm	Reward	Observation	Environment
1	PPO	base	raw $[\theta, \omega]$	nominal
2	PPO	tuned	raw $[\theta, \omega]$	nominal
3	PPO	base	$[\sin \theta, \cos \theta, \omega]$	nominal
4	PPO	tuned	$[\sin \theta, \cos \theta, \omega]$	nominal
5	PPO	base	$[\sin \theta, \cos \theta, \omega]$	robust
6	PPO	tuned	$[\sin \theta, \cos \theta, \omega]$	robust
7	A2C	base	raw $[\theta, \omega]$	nominal
8	A2C	tuned	raw $[\theta, \omega]$	nominal
9	A2C	base	$[\sin \theta, \cos \theta, \omega]$	nominal
10	A2C	tuned	$[\sin \theta, \cos \theta, \omega]$	nominal
11	A2C	base	$[\sin \theta, \cos \theta, \omega]$	robust
12	A2C	tuned	$[\sin \theta, \cos \theta, \omega]$	robust

F. Training

Both A2C and PPO are trained using the Stable-Baselines3 implementation with an MLP policy. The same training procedure and early-stopping logic are used for both algorithms, so that the comparison focuses on the algorithm and the configuration choices.

Each model is trained for a maximum of 10^6 environment steps. Every 25,000 steps, an evaluation callback evaluates the current policy. Training stops if the evaluation return reaches approximately 180, or if no new best evaluation return is found after three consecutive evaluation callbacks. The first condition stops training once the policy has essentially solved the swing-up task. The second condition avoids wasting computation on runs that appear to be stuck in a local optimum.

After training, the final model for each seed and configuration evaluated over 10 deterministic episodes. The final results are then averaged over the six seeds for each configuration.

G. Results

The multi-seed results are reported in Table VIII. Overall, PPO performs much better than A2C. Across all configurations, PPO reaches an average mean reward of 172.6, while A2C reaches only 66.0. The paired comparison across matched seeds confirms this difference. PPO has an average advantage of +106.6 reward over A2C, with a Wilcoxon signed-rank test of $p = 2.91 \cdot 10^{-11}$. PPO is also more stable across seeds, with a standard deviation of 12.3 compared with 36.1 for A2C.

The observation representation has a clear effect on PPO. PPO with sine-cosine observations gives the best results, especially in the nominal environment. Compared with raw observations, the sine-cosine representation improves PPO by +23.9 reward on average. This suggests that removing the angular wrap-around discontinuity makes the learning problem easier.

TABLE VIII: Swing-up training results over six seeds per configuration. “Mean reward” is the mean return over 10 evaluation episodes, averaged across the six seeds. “Steps” and “Time” are also reported as averages across the six seeds. Configuration is reported from the flags actually passed to training.

Algo	Reward	Obs	Dist.	Steps	Time [min]	Mean reward
PPO	base	raw	nom.	496k	5.4	163.4 ± 5.3
PPO	tuned	raw	nom.	442k	4.1	150.8 ± 5.9
PPO	tuned	sincos	nom.	167k	1.4	181.1 ± 0.7
PPO	base	sincos	nom.	192k	1.8	180.8 ± 0.4
PPO	tuned	sincos	rob.	342k	3.2	178.3 ± 4.5
PPO	base	sincos	rob.	233k	2.0	181.1 ± 0.3
A2C	base	raw	nom.	371k	10.8	69.7 ± 31.3
A2C	tuned	raw	nom.	367k	16.5	85.2 ± 28.6
A2C	base	sincos	nom.	350k	13.7	39.3 ± 30.7
A2C	tuned	sincos	nom.	350k	12.1	49.7 ± 35.6
A2C	base	sincos	rob.	492k	11.4	84.7 ± 28.4
A2C	tuned	sincos	rob.	462k	10.7	67.2 ± 47.6

The reward formulation has a smaller effect. PPO with the base reward gives a slightly higher final reward than PPO with the tuned reward on average. However, this does not mean that the base reward is always the better practical choice. Several PPO sine-cosine configurations reach the reward ceiling of about 180, so their final rewards are almost the same. In that case, the number of steps needed to reach the threshold is more informative. Using this criterion, the fastest reliable controller is PPO with tuned reward, sine-cosine observations, and nominal training. It reaches the threshold in all six seeds after about 167k steps on average. The corresponding base-reward sine-cosine nominal controller also reaches the threshold in all six seeds, but needs about 192k steps.

Robust training does not strongly reduce the final PPO reward, but it does increase the training cost. The robust PPO controllers can still solve the swing-up task, which is promising for transfer to the physical setup. However, they generally need more training steps. With the tuned reward, robust training also reduces reliability, since only four out of six seeds reach the reward threshold. Therefore, robust training should be seen as a trade-off: it may make the controller more tolerant to actuator noise and dropped actions, but it is not free in terms of training efficiency.

Based on the results, A2C is not competitive for this task. Its best configurations remain far below the best PPO configurations, and its results vary much more between seeds. Visual inspection of the trained policies supports this conclusion. Several A2C agents either fail to generate a reliable swing-up or fail to stabilise the disk after reaching the upper part of the trajectory. Thus, the preferred controller for nominal simulation is PPO with sine-cosine observations and tuned reward. If robustness is more important for the physical setup, the robust PPO sine-cosine controller remains a possible choice, but it requires more training steps.

V. REFERENCE TRACKING

A. Approach

As shown in Section IV-G, PPO outperforms A2C across all evaluated configurations, and several configurations managed to adopt a working policy — most notably some of those trained and evaluated in the *robust* environment. Moving forward, we therefore extend the task from pure balancing to *reference tracking*, using our most capable configuration: SinCos observations with the base reward. Furthermore we train and evaluate in the robust environment.

For reference tracking the base reward has been reshaped so that it is highest at a desired reference angle measured away from the upright position, rather than at the upright position itself. To this end we added a tracking reward (`tracking_reward`) and a tracking flag to the environment, together with an optional angle parameter that specifies the desired offset from upright in degrees. The flag enables reference tracking at varying angles, re-centering the reward so that its maximum lies at the specified angle away from the top rather than at the top itself.

Let θ_{ref} denote the target angle, i.e. the upright position offset by the desired reference angle. The signed angular distance to this target, wrapped to $[-\pi, \pi]$, is

$$\Delta_{\text{ref}}(\theta) = \text{atan2}(\sin(\theta - \theta_{\text{ref}}), \cos(\theta - \theta_{\text{ref}})). \quad (11)$$

The tracking reward is then a cosine bowl centered on the target, with an optional sharp bonus around it,

$$r_{\text{track}}(\theta) = \frac{1 + \cos \Delta_{\text{ref}}(\theta)}{2} + 0.5 \beta \exp\left(-\left(\frac{\Delta_{\text{ref}}(\theta)}{0.25}\right)^2\right), \quad (12)$$

where the cosine term peaks at 1.0 when $\theta = \theta_{\text{ref}}$. The tuned variant ($\beta = 1$), again adds the gaussian bonus introduced by 9. This using this equation we can also distinguish between + and -15 degrees.

B. Results

Our approach yielded PPO models that were able to keep balance up to a reference angle of 20 degrees. Moving up to 25 degrees, we found that the policy never learns the swing-up anymore, therefore getting stuck at the boundary that the maximum gain is able to push towards (Figure 8). This limitation could be countered by introducing the reference angle in the observation and then training with an increasing reference angle over episodes. The agent would then learn the swing-up during high-angle episodes and find the relation between the angle observation and the reward over time.

TABLE IX: Reference-tracking results for the PPO controller.

Reference angle	Steps	Time [min]	Mean reward
15°	950k	11.3	177.3
20°	1001k	12.0	177.6
25°	775k	10.8	61.7
-15°	1001k	10.3	175.9

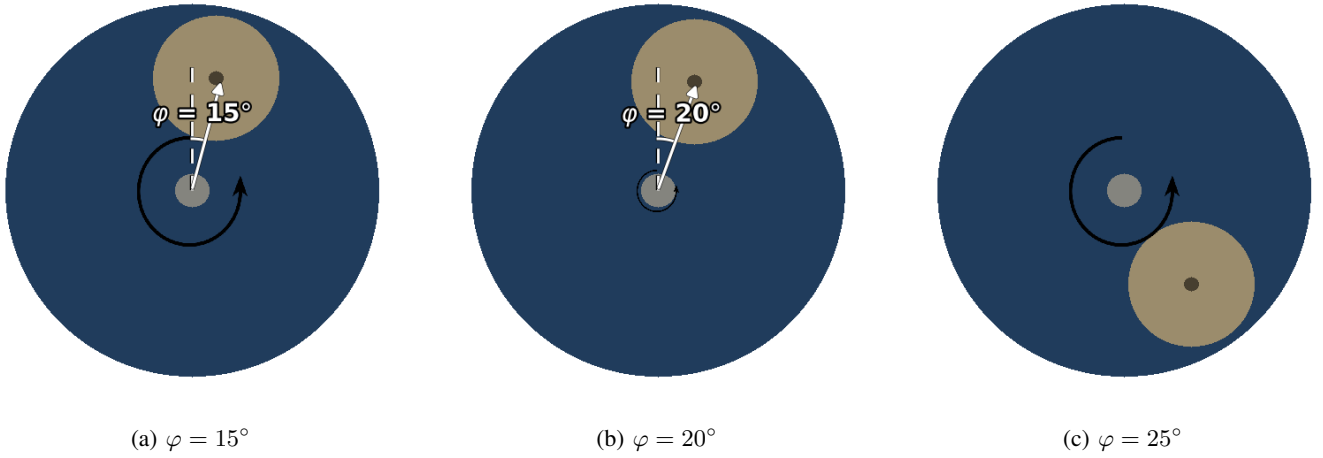


Fig. 8: Disk setups at increasing reference angles φ . (c) 25 degrees, policy loses swing-up learning capability.

REFERENCES

- [1] A. Dong, A. Starr, and Y. Zhao, “Neural network-based parametric system identification: A review,” *International Journal of Systems Science*, vol. 54, no. 13, pp. 2676–2688, 2023. DOI: 10.1080/00207721.2023.2241957.
- [2] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994. DOI: 10.1109/72.279181.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.
- [4] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems*, vol. 24, Curran Associates, Inc., 2011, pp. 2546–2554.